

Rapport de mission
technique
2eme année
Amélioration du processus R&D



Yassine Hamrouni

Tuteur entreprise : Maxime Phan Thanh

Tuteur école : Johan Thapper

Sommaire

Introduction.....	1
I. Présentation générale de l'entreprise.....	2
II. Évolution socio-économique.....	2
1. Croissance internationale et diversification sectorielle.....	2
2. Impact sur l'emploi et les compétences.....	2
3. Contributions à la compétitivité industrielle.....	3
4. Responsabilité sociétale et développement durable.....	3
III – Présentation d'équipe.....	4
1. Service d'accueil.....	4
1.1. Missions principales.....	4
1.2 Objectifs du processus.....	4
1.3 Importance et accessibilité.....	4
2. Organigramme de l'équipe.....	5
2.1 Structure générale.....	5
2.2 Mon rôle spécifique.....	5
IV. Mes missions.....	6
1. Contexte et Enjeux.....	6
Enjeux pour l'entreprise.....	6
2. Objectifs opérationnels.....	7
3. Moyens et Ressources.....	8
Backend : Node.js + Express.....	8
Frontend : React.....	8
Base de données : PostgreSQL.....	8
4. Déroulement et Méthodologie.....	10
4.1 Mise en place du middleware d'authentification.....	10
4.1.1 Objectif.....	10
4.1.2 Principe.....	10
4.1.3 Déroulement opérationnel — flux logique du middleware.....	10
4.1.4 Comportement et réponses HTTP standards.....	11
4.1.5 Sécurité et bonnes pratiques.....	11
4.2 Intégration du CAS (Central Authentication Service).....	13
4.2.1 Objectif.....	13
4.2.2 Flux fonctionnel (vue d'ensemble).....	13
4.2.3 Exemple de logique (en langage courant).....	14
4.2.4 Gestion des erreurs et cas particuliers.....	14
4.2.5 Sécurité et bonnes pratiques spécifiques au CAS.....	14
4.2.6 Tests et exploitation.....	15
4.3 Sécurité des données.....	15

4.3.1 Objectif.....	16
4.3.2 Chiffrement et gestion des tokens.....	16
4.3.3 HTTPS obligatoire.....	17
4.3.4 Contrôles d'accès — trigrammes et autorisation.....	17
4.3.5 Journalisation et traçabilité.....	17
4.3.6 Mesures d'atténuation et bonnes pratiques.....	18
4.4 Développement du Frontend et gestion des événements.....	18
4.5 Gestion des événements et des données.....	20
4.5.1 Objectif.....	20
4.5.2 Source de vérité : l'URL et la synchronisation d'état.....	20
4.5.3 Cycle de vie d'un chargement (schéma useEffect).....	20
4.5.4 Annulation, concurrence et cohérence.....	21
4.5.5 Gestion des états UI et indicateurs.....	21
4.5.6 Cache court et stratégies de fetching.....	21
4.5.7 Pagination, scroll infini et UX.....	21
4.5.8 Comportement spécifique : logs d'actions (events).....	22
4.5.9 Filtres, recherches et tris côté client (sans reload).....	22
4.5.10 Sécurité et résilience des appels.....	23
V. Bilan professionnel.....	25
VI. Conclusion.....	26
Bibliographie.....	27
Résumé.....	28
Résumé anglais.....	30

Glossaire

API (Application Programming Interface)

Interface qui permet à différentes applications de communiquer entre elles via des services ou des fonctions.

EN — API (Application Programming Interface): Interface that allows different applications to communicate with each other via services or functions.

Backend

Partie d'une application qui gère la logique métier, la base de données et les échanges avec le serveur.

EN — Backend: The part of an application that handles business logic, the database and server-side interactions.

Base de données relationnelle

Système de stockage des données sous forme de tables reliées entre elles par des relations (ex. PostgreSQL).

EN — Relational database: A system that stores data in tables linked by relationships (e.g. PostgreSQL).

CAS (Central Authentication Service)

Protocole d'authentification unique (Single Sign-On) permettant de se connecter une seule fois pour accéder à plusieurs applications sécurisées.

EN — CAS (Central Authentication Service): A single sign-on protocol enabling one-time login to access multiple secure applications.

Certification logicielle

Processus de validation garantissant que les logiciels fonctionnent correctement sur des configurations matérielles spécifiques.

EN — Software certification: Process that validates software operates correctly on specific hardware configurations.

CRUD (Create, Read, Update, Delete)

Ensemble des quatre opérations de base pour manipuler les données dans une base.

EN — CRUD (Create, Read, Update, Delete): The four basic operations to manipulate data in a database.

Excel

Logiciel tableur de Microsoft, utilisé pour manipuler et stocker des données sous forme de fichiers.

EN — Excel: Microsoft spreadsheet software used to manipulate and store data in files.

Express

Framework minimaliste pour Node.js, utilisé pour créer des serveurs web et des API REST.

EN — Express: Minimalist framework for Node.js used to build web servers and REST APIs.

Frontend

Partie visible d'une application web, correspondant à l'interface utilisateur.

EN — Frontend: The visible part of a web application—the user interface.

GitLab

Plateforme de gestion de version et de CI/CD permettant de stocker du code, collaborer et automatiser le déploiement.

EN — GitLab: Version control and CI/CD platform for hosting code, collaboration and automated deployment.

IDE (Integrated Development Environment)

Environnement de développement intégré regroupant des outils pour coder (éditeur, débogueur, compilateur).

EN — IDE (Integrated Development Environment): Development environment bundling tools to code (editor, debugger, compiler).

Journalisation (Logs)

Enregistrement automatique des actions effectuées sur une application (qui, quoi, quand).

EN — Logging (Logs): Automatic recording of actions in an application (who, what, when).

Jumeau numérique

Représentation virtuelle d'un produit ou processus permettant de le simuler et de l'optimiser.

EN — Digital twin: Virtual representation of a product or process for simulation and optimization.

KAO

Terme interne désignant une ou plusieurs erreurs remontées lors des tests logiciels (acronyme spécifique à l'organisation).

EN — KAO: Internal term for one or more errors reported during software tests (organization-specific acronym).

Maintenance prédictive

Méthode qui anticipe les défaillances grâce aux données recueillies, afin d'optimiser les interventions.

EN — Predictive maintenance: Method that anticipates failures using collected data to optimize interventions.

Middleware

Composant logiciel intermédiaire qui gère les communications ou traitements avant qu'une requête n'atteigne sa destination (ex. authentication).

EN — Middleware: Software layer that handles communications or processing before a request reaches its destination (e.g. authentication).

Ngrok

Outil permettant d'exposer un serveur local sur Internet via une URL temporaire sécurisée en HTTPS.

EN — Ngrok: Tool to expose a local server to the Internet via a temporary HTTPS URL.

Node.js

Environnement d'exécution JavaScript côté serveur.

EN — Node.js: JavaScript runtime environment on the server side.

ODT (tests logiciels)

Tests effectués par l'équipe software pour vérifier le bon fonctionnement des logiciels sur différentes configurations matérielles.

EN — ODT (software tests): Tests run by the software team to verify correct operation across hardware configurations.

ORM (Object-Relational Mapping)

Technique qui permet de manipuler une base de données avec un langage orienté objet (ex. Sequelize).

EN — ORM (Object-Relational Mapping): Technique to manipulate a database using object-oriented code (e.g. Sequelize).

PLM (Product Lifecycle Management)

Gestion du cycle de vie d'un produit, de sa conception à sa maintenance.

EN — PLM (Product Lifecycle Management): Managing a product's lifecycle from design to maintenance.

PostgreSQL

Système de gestion de base de données relationnelle open-source, robuste et performant.

EN — PostgreSQL: Open-source, robust and high-performance relational database management system.

React

Bibliothèque JavaScript utilisée pour construire des interfaces utilisateur dynamiques.

EN — React: JavaScript library used to build dynamic user interfaces.

R&D (Recherche et Développement)

Activité de conception et d'innovation dans les entreprises technologiques.

EN — R&D (Research & Development): Design and innovation activities in tech companies.

Sequelize

ORM pour Node.js, facilitant l'interaction avec des bases de données relationnelles comme PostgreSQL.

EN — Sequelize: ORM for Node.js that simplifies interaction with relational databases like PostgreSQL.

SSO (Single Sign-On)

Système d'authentification unique permettant d'accéder à plusieurs services après une seule connexion.

EN — SSO (Single Sign-On): Authentication system allowing access to multiple services after a single login.

Token

Jeton d'authentification utilisé pour sécuriser les communications entre un client et un serveur.

EN — Token: Authentication token used to secure communication between a client and a server.

Workstation

Station de travail informatique dédiée à des tâches exigeantes (ex. conception 3D, simulation).

EN — Workstation: Computer workstation dedicated to demanding tasks (e.g. 3D design, simulation).

XML

Format de données structuré, souvent utilisé pour l'échange d'informations entre systèmes.

EN — XML: Structured data format often used for information exchange between systems.

xml2js

Bibliothèque JavaScript permettant de convertir du XML en objets JavaScript.

EN — xml2js: JavaScript library to convert XML into JavaScript objects.

Introduction

Au cours de ma troisième année d'apprentissage chez Dassault Systèmes, à Vélizy-Villacoublay, j'ai poursuivi la mission entamée l'année précédente dans le cadre de mon poste d'apprenti ingénieur informatique. Après avoir conçu une base de données et initié le développement d'une API sécurisée, j'ai pu, durant cette nouvelle période, finaliser l'API et proposer une première version fonctionnelle du site permettant son exploitation.

Cette année a été marquée par une progression significative dans la mise en place d'une solution complète visant à remplacer l'usage des fichiers Excel au sein de l'équipe hardware. Mon travail s'est articulé autour de plusieurs axes :

- **Finalisation et optimisation de l'API**, afin d'améliorer sa stabilité, sa sécurité et ses performances.
- **Développement d'une première interface web**, intuitive et ergonomique, permettant aux membres de l'équipe de tester et d'exploiter facilement les fonctionnalités de l'API.
- **Mise en place d'outils de validation et de tests**, pour garantir la fiabilité de l'ensemble du système.

Ce rapport s'attachera dans un premier temps à présenter l'évolution de la mission et les améliorations apportées à l'API. Il détaillera ensuite la conception et la mise en œuvre de la première version du site, en soulignant les choix techniques réalisés ainsi que les défis rencontrés. Enfin, il se conclura par un bilan personnel de cette deuxième année d'apprentissage, mettant en avant les compétences acquises et les perspectives pour la suite du projet.

I. Présentation générale de l'entreprise

Fondée en 1981 comme filiale logicielle de Dassault Aviation, **Dassault Systèmes** est rapidement devenue un leader mondial des solutions de conception 3D, de maquettes numériques et de gestion du cycle de vie produit (PLM).

Basée à **Vélizy-Villacoublay (Île-de-France)**, l'entreprise emploie aujourd'hui plus de **20 000 collaborateurs** dans plus de **140 pays**, au service de centaines de milliers de clients issus de multiples secteurs industriels.

Mission : accompagner la transformation digitale des entreprises en proposant des outils innovants pour **concevoir, simuler et optimiser** produits et processus, de l'aéronautique aux biens de consommation.

II. Évolution socio-économique

1. Croissance internationale et diversification sectorielle

- **Années 1980-1990 :**
Démarrage centré sur l'aéronautique, adoption rapide par l'automobile et l'énergie.
 - **Années 2000 :**
Expansion en Asie-Pacifique et Amériques ; acquisitions stratégiques (ex. *SolidWorks* en 1997) pour enrichir l'offre PLM.
 - **Aujourd'hui :**
Présence renforcée dans la santé, la construction, les services publics et les biens de consommation, grâce à la plateforme **3DEXPERIENCE**.
-

2. Impact sur l'emploi et les compétences

- Création de **milliers d'emplois qualifiés** (R&D, ingénierie logicielle, services), moteur de l'essor des métiers du numérique.
- Développement de **partenariats avec universités et écoles d'ingénieurs** pour former la main-d'œuvre aux technologies 3D, simulation et data science.

3. Contributions à la compétitivité industrielle

- **Automatisation et optimisation** des processus de conception et fabrication → réduction des délais et coûts de développement.
- Déploiement de la **maintenance prédictive** et des **jumeaux numériques**, améliorant la performance opérationnelle et la durabilité.

4. Responsabilité sociétale et développement durable

- Engagements pour **réduire l'empreinte carbone**, via la simulation énergétique des bâtiments et l'optimisation des réseaux.
- Initiatives pour l'**inclusion numérique** : programmes de formation et licences éducatives accessibles dans les pays en développement.

III – Présentation d'équipe

1. Service d'accueil

Mon service d'accueil chez **Dassault Systèmes** est spécialisé dans la **certification des logiciels selon différentes configurations matérielles**.

1.1. Missions principales

- Tester les logiciels sur diverses combinaisons de :
 - processeurs, cartes graphiques (GPU), systèmes d'exploitation,
 - versions de logiciels, environnements de développement (IDE) et langages (ex. C++).
- Réaliser chaque année un **processus rigoureux de certification** des machines utilisées par nos clients et nos collaborateurs internes.

1.2 Objectifs du processus

- Garantir la **compatibilité et la performance** des logiciels sur les configurations testées.
- Produire un document annuel appelé “**define**”, qui compile les résultats et évalue l'aptitude des machines à exécuter nos solutions.
- Fournir aux clients et partenaires des **directives claires sur les configurations matérielles recommandées** pour une performance optimale.

1.3 Importance et accessibilité

- La certification contribue à **assurer la fiabilité et la qualité** des produits Dassault Systèmes.
- Elle renforce la **confiance des clients** dans nos solutions.

- Les documents de certification sont disponibles sur le site officiel, via la section **Support Hardware and Software**.
-

2. Organigramme de l'équipe

2.1 Structure générale

L'équipe est divisée en deux branches principales :

- **Hardware :**
 - Réception et gestion du matériel (machines complètes ou composants, ex. cartes graphiques).
 - Stockage dans un vaste pool de plusieurs centaines de machines en fonctionnement permanent, permettant des tests continus.
- **Software :**
 - Réalisation des tests logiciels appelés **ODT**.
 - Détection et remontée des erreurs (appelées **KAO**).
 - Correction des anomalies en interne ou collaboration avec les développeurs concernés.

2.2 Mon rôle spécifique

Je participe à l'amélioration du **processus de certification côté hardware**.

- Situation actuelle : chaque nouvelle machine est enregistrée dans un fichier Excel, une méthode peu efficace pour retrouver rapidement une machine.
- Solution proposée : développement d'un **site web interne** permettant de localiser chaque machine à l'aide d'un **code-barres**.
- Objectifs :
 - Simplifier la gestion du parc de machines,

- Accélérer la recherche et la récupération des informations,
- Améliorer globalement l'efficacité et la fiabilité du processus de certification.

IV. Mes missions

1. Contexte et Enjeux

L'équipe hardware de Dassault Systèmes utilise depuis plusieurs années des fichiers Excel pour gérer la certification de ses composants et matériels. Bien que pratique pour des volumes réduits, cette solution est devenue inadaptée :

- Les fichiers étaient **multipliés** et difficiles à maintenir.
- Les **erreurs humaines** (copier-coller, doublons, suppression involontaire) impactaient la fiabilité des données.
- L'absence de **sécurité avancée** exposait les données à des accès non autorisés.
- Les **processus manuels** (envoi de fichiers par email, versions locales) ralentissaient la productivité.

Enjeux pour l'entreprise

- **Sécuriser les données** de certification (critiques pour la conformité qualité et légale).
- **Améliorer la productivité** des équipes en centralisant l'information.
- **Réduire les erreurs** liées à la manipulation manuelle d'Excel.
- **Standardiser les accès** via un système unique d'authentification (CAS, déjà utilisé par Dassault Systèmes).
- **Préparer une architecture évolutive** permettant l'intégration avec d'autres systèmes internes.

2. Objectifs opérationnels

Les objectifs de ma mission ont été formulés en termes d'outils et de technologies concrètes :

- **Base de données PostgreSQL : migration des données Excel → base relationnelle robuste.**
- **API REST en Node.js/Express : gestion CRUD des certifications (Create, Read, Update, Delete).**
- **Middleware de sécurité : intégration avec CAS + génération de tokens.**
- **Validation des données : regex pour emails et téléphones, contraintes d'unicité dans PostgreSQL.**
- **Interface web (frontend) : simplification des tests API et mise à disposition des données.**
- **Journalisation : logs pour suivre toutes les actions sur l'API.**

3. Moyens et Ressources

- **Langages & frameworks :**

Backend : Node.js + Express

- **Node.js** : environnement d'exécution JavaScript côté serveur, idéal pour développer des applications performantes et scalables. Son modèle asynchrone et non bloquant permet de gérer efficacement un grand nombre de requêtes simultanées.
- **Express** : framework minimaliste basé sur Node.js, facilitant la création et la gestion des routes, middlewares et API REST. Il offre une structure claire pour la communication entre le front-end et la base de données.

Frontend : React

- **Utilisé pour développer le prototype du site web.**
- **React permet de créer une interface utilisateur dynamique et réactive grâce à son système de composants réutilisables.**
- **Son approche "state management" rend l'affichage des informations (par exemple la localisation des machines) rapide et fluide.**

Base de données : PostgreSQL

- **Base de données relationnelle robuste et open-source.**
- **Particulièrement adaptée pour gérer des données structurées et complexes (ex. inventaire des machines avec leurs configurations, emplacements et statuts).**
- **Supporte des transactions fiables et sécurisées, garantissant l'intégrité des données dans le processus de certification.**

- **Bibliothèques clés :**

- **axios** : utilisée pour la communication entre le front-end et le système d'authentification CAS (Central Authentication Service). Permet d'envoyer facilement des requêtes HTTP et de gérer les réponses.
- **xml2js** : bibliothèque servant à parser les réponses CAS reçues au format XML, afin de les convertir en objets JavaScript exploitables.
- **crypto** : module natif Node.js utilisé pour la génération et le chiffrement des tokens de sécurité. Garantit l'authentification et la protection des échanges de données.
- **sequelize** : ORM (Object-Relational Mapping) permettant de manipuler PostgreSQL avec du code JavaScript. Facilite les opérations CRUD (Create, Read, Update, Delete) et la gestion des relations entre tables.
-
- **Outils utilisés :**
 - **Postman** : outil de test d'API utilisé pour vérifier la validité des routes backend et simuler des scénarios d'utilisation avant intégration front-end.
 - **GitLab** : gestion de version du code source, avec mise en place de CI/CD (Intégration Continue / Déploiement Continu) afin d'automatiser les tests et déploiements.
 - **Ngrok** : solution pour exposer temporairement le serveur backend en HTTPS, permettant de tester facilement l'intégration entre le front-end et le back-end sans configuration complexe de réseau.

4. Déroulement et Méthodologie

4.1 Mise en place du middleware d'authentification

4.1.1 Objectif

L'objectif du middleware d'authentification est d'empêcher tout accès non autorisé aux routes sensibles de l'API en jouant le rôle d'un filtre placé en amont de la logique métier. Concrètement, ce composant vérifie l'identité de l'émetteur de la requête, contrôle l'intégrité et la validité du jeton présenté, charge les informations utilisateur nécessaires et prend la décision d'accepter ou de rejeter la requête avant que le code métier ne s'exécute. Ce contrôle centralisé garantit une application cohérente des règles de sécurité et simplifie la mise en place de mécanismes complémentaires (logging, audit, révocation).

4.1.2 Principe

On peut comparer le middleware à un agent de contrôle d'accès : à chaque requête qui souhaite accéder à une ressource protégée on demande la présentation d'un « badge » — le token. Le middleware s'assure que ce badge est bien présent, qu'il est authentique (il n'a pas été falsifié) et qu'il n'a pas été révoqué. Si toutes ces conditions sont remplies, la requête est transmise à l'application avec le contexte utilisateur attaché ; sinon, l'accès est refusé et une réponse d'erreur est renvoyée au client.

4.1.3 Déroulement opérationnel — flux logique du middleware

Au niveau fonctionnel, le middleware suit une séquence de vérifications qui commence dès la réception de la requête. Il extrait d'abord l'en-tête d'autorization et vérifie que le format est conforme (typiquement « Bearer <token> »). Si l'en-tête est absent ou malformé, la requête est immédiatement rejetée avec un code 401 et un message clair indiquant l'absence ou le mauvais format du token. Lorsque le token est présent, le middleware effectue une vérification syntaxique minimale pour filtrer les valeurs manifestement invalides, puis procède à la validation cryptographique : pour un JWT, il contrôle la signature et la date d'expiration à l'aide de la clé connue ; pour un token opaque, il interroge la source d'autorité (base de données ou service d'authentification) pour confirmer la validité. Si la validation cryptographique échoue, la requête reçoit une réponse 401 indiquant que le token est invalide ou expiré.

En complément, il est fortement recommandé d'ajouter une étape de vérification de révocation : le middleware consulte alors une liste de tokens révoqués (par exemple une blacklist stockée dans Redis) ou la table de sessions pour s'assurer que le token présenté n'a pas été invalidé (par un logout, une compromission, etc.). Si le token a été révoqué, l'accès est refusé.

Une fois le token validé, le middleware récupère l'identifiant utilisateur contenu dans le payload du token (ou lié à la session) et charge l'objet utilisateur depuis la base de données. Si l'utilisateur a été supprimé ou désactivé, la requête est rejetée. Lorsque l'utilisateur est chargé avec succès, le middleware injecte ce contexte dans la requête (par exemple `req.user` ou équivalent), ce qui permet aux handlers suivants d'accéder aux informations nécessaires pour appliquer la logique métier. Enfin, si la route cible impose des contrôles d'autorisation supplémentaires (vérification d'un rôle ou d'une permission), ces contrôles sont effectués et, en cas de défaut, un code 403 est retourné. Si toutes les vérifications sont satisfaites, le middleware appelle `next()` pour laisser la requête poursuivre son traitement.

4.1.4 Comportement et réponses HTTP standards

Les réponses renvoyées par le middleware suivent des conventions HTTP classiques : un code 401 (Unauthorized) signale l'absence d'identification ou un jeton invalide/expiré ; un code 403 (Forbidden) indique que l'utilisateur est authentifié mais n'a pas les droits requis pour l'action demandée ; un code 400 (Bad Request) est utilisé pour un header mal formé ou une requête incorrecte ; enfin un code 500 (Internal Server Error) signale une erreur serveur imprévue, par exemple un problème d'accès à la base. Dans tous les cas, les messages retournés doivent rester clairs pour l'utilisateur ou le client API tout en évitant de divulguer des informations techniques sensibles (par exemple préférer « identifiants invalides » à « signature invalide » en production).

4.1.5 Sécurité et bonnes pratiques

Pour garantir la robustesse du mécanisme, plusieurs bonnes pratiques sont à respecter. Les communications avec l'API doivent être systématiquement chiffrées (HTTPS/TLS) afin d'empêcher l'interception des tokens en transit. Les clés et secrets utilisés pour signer ou chiffrer les tokens doivent être conservés dans un gestionnaire de secrets (KMS, Vault) ou des variables d'environnement sécurisées, et faire l'objet d'une politique de rotation. La stratégie recommandée consiste à utiliser des access tokens de courte durée (par exemple 15 à 60 minutes) et des refresh tokens de plus longue durée stockés et protégés côté serveur (ou via des cookies `HttpOnly`), afin de limiter l'impact d'un vol de token. Il est également conseillé d'éviter d'inscrire des données sensibles dans le payload d'un JWT et

de prévoir un mécanisme d'invalidation (blacklist ou table de sessions) pour pouvoir révoquer un token avant son expiration. Enfin, toute mise en œuvre doit s'accompagner de tests (unitaires et d'intégration), de logs et d'un monitoring adaptés pour détecter rapidement les anomalies et incidents liés à l'authentification.

Toutes les routes sensibles passent par le middleware suivant :

```
const authentication = async (req, res, next) => {
  try {
    const authToken = req.header('Authorization')?.replace('Bearer ', '');
    if (!authToken) {
      return res.status(401).json({ message: "Token manquant." });
    }

    const encryptedToken = encrypt(authToken);
    const user = await Ds_user.findOne({ where: { token: encryptedToken } });

    if (!user) {
      return res.status(401).json({ message: "Token invalide ou expiré." });
    }

    req.user = user; // utilisateur injecté dans la requête
    next();
  } catch (error) {
    res.status(500).json({ message: "Erreur d'authentification." });
  }
};
```

Chaque appel API est ainsi filtré et validé avant d'être traité.

4.2 Intégration du CAS (Central Authentication Service)

Le Central Authentication Service (CAS) est un protocole d'authentification unique (Single Sign-On – SSO) permettant à un utilisateur de se connecter une seule fois et d'accéder à plusieurs applications sécurisées sans devoir ressaisir ses identifiants.

L'intégration CAS permet aux utilisateurs de se connecter une seule fois (SSO) pour accéder à toutes les applications internes. Techniquement, après la connexion le serveur CAS délivre un « ticket » que notre application vérifie côté serveur auprès du CAS. Si la vérification est positive, nous créons une session interne sécurisée (token) pour l'utilisateur. Cela centralise la gestion des comptes, simplifie l'expérience utilisateur et renforce la sécurité (authentification centralisée, gestion des sessions et possibilités de Single Logout).

4.2.1 Objectif

Permettre à un utilisateur de s'authentifier une fois via le serveur CAS et d'accéder ensuite à plusieurs applications internes sans ressaisir ses identifiants. Le backend valide les « tickets » émis par CAS, traduit le résultat en une session interne (token) et fournit une méthode standardisée pour toutes les requêtes ultérieures (ex. `Authorization: Bearer <token>`).

4.2.2 Flux fonctionnel (vue d'ensemble)

Le flux fonctionnel commence dès que l'utilisateur tente d'accéder à l'application A. L'application détecte qu'aucune session locale n'existe et redirige l'utilisateur vers la page de connexion du serveur CAS, en passant le paramètre `service` correspondant à l'URL de retour. Après saisie de ses identifiants sur le serveur CAS, l'utilisateur est renvoyé vers l'URL de service de l'application accompagnée d'un paramètre `ticket`. À ce stade le client ne décide rien : c'est le backend de l'application qui prend le relais. Il récupère le ticket reçu et appelle l'endpoint de validation du CAS (`serviceValidate` ou `p3/serviceValidate`) pour obtenir la confirmation d'authentification et, le cas échéant, les attributs associés (identifiant, email, rôles, etc.). Si CAS confirme l'authentification, le backend crée ou met à jour la session utilisateur interne, émet un token interne (JWT signé ou token opaque stocké) et le transmet au client — soit via un cookie `HttpOnly`, soit via un header `Authorization`, selon les contraintes d'utilisation. Les appels suivants au backend présentent ce token : le middleware d'authentification interne le valide et injecte le contexte utilisateur dans chaque requête.

4.2.3 Exemple de logique (en langage courant)

Quand le backend reçoit un ticket, il construit l'URL de validation en reproduisant exactement le paramètre **service** utilisé lors de la redirection initiale, puis appelle CAS en HTTPS. La réponse de CAS (généralement en XML ou JSON selon la configuration) est parsée ; si elle contient une balise **authenticationSuccess** ou l'équivalent, le backend extrait l'identifiant et les attributs fournis. Ensuite, l'application recherche un utilisateur local correspondant (par identifiant CAS ou par email) : si l'utilisateur existe, il est mis à jour si nécessaire ; s'il n'existe pas, la politique de provisioning détermine s'il faut créer un compte minimal ou refuser l'accès. Enfin, le backend émet un token interne contenant les informations utiles à notre application (user id, rôles, jti, expiration), enregistre la session (ou le jti) si l'on veut pouvoir révoquer ultérieurement, et redirige l'utilisateur vers l'URL d'origine. Le token interne est alors utilisé par le client pour toutes les requêtes suivantes, ce qui permet au middleware interne d'appliquer l'autorisation sans devoir interroger CAS à chaque appel.

4.2.4 Gestion des erreurs et cas particuliers

Plusieurs cas doivent être traités explicitement pour assurer robustesse et sécurité. Si le ticket est absent ou manifestement invalide, l'utilisateur reçoit un message l'invitant à retenter la connexion ; il ne faut jamais accepter un ticket depuis le client sans validation serveur. Si le serveur CAS est indisponible, l'application doit présenter un message de dégradation clair (ex. « service d'authentification momentanément indisponible ») et logger l'incident ; selon la criticité, on peut bloquer l'accès aux pages protégées ou proposer un mode dégradé pour les pages publiques. Si les attributs retournés par CAS sont insuffisants (absence d'email ou d'attribut(s) requis), la politique peut consister à refuser l'accès ou à lancer un workflow de complétion où l'utilisateur complète les informations manquantes. En cas de tentative de réutilisation d'un ticket (replay), le backend doit refuser et consigner l'événement : un ticket CAS est à usage unique et toute tentative de réutilisation est suspecte. Enfin, si le paramètre **service** retourné ne correspond pas à l'URL attendue, la validation doit être rejetée pour prévenir tout détournement par redirection.

4.2.5 Sécurité et bonnes pratiques spécifiques au CAS

Plusieurs précautions sont indispensables. Les échanges avec le serveur CAS doivent toujours se faire en HTTPS avec validation stricte du certificat ; jamais le ticket côté client ne doit être considéré comme preuve d'authentification sans vérification serveur. La valeur **service** doit être validée strictement et ne doit accepter que les URL d'applications pré-enregistrées chez le CAS, afin d'éviter les open redirects. Du côté applicatif, il est recommandé de stocker les sessions et tokens de façon sécurisée : les tokens internes courts (access tokens) doivent être signés et avoir une durée limitée,

tandis que la capacité de révoquer (blacklist, table sessions) doit être disponible pour gérer les logouts et les compromissions. Les clés et secrets de signature/chiffrement doivent être gérés dans un KMS ou un vault et faire l'objet d'une rotation contrôlée. La journalisation doit inclure des informations utiles (userId, identification partielle du ticket, IP, horodatage) sans jamais inscrire les tokens en clair dans les logs. Si l'environnement l'exige, implémenter et tester le Single Logout (SLO) pour que les notifications de logout du CAS invalident correctement les sessions locales. Enfin, protéger les endpoints d'authentification par des mécanismes de limitation de débit et d'anti-brute force, et prévoir des timeouts et des stratégies de retry/backoff lors des appels vers CAS.

4.2.6 Tests et exploitation

Pour garantir la qualité, prévoir des tests unitaires simulant des réponses CAS (succès et échecs), ainsi que des tests d'intégration couvrant le parcours complet (redirection → login CAS → retour avec ticket → validation → session créée). En exploitation, monitorer la latence et le taux d'erreurs des appels à `serviceValidate`, alerter sur les réutilisations de tickets ou les pics d'échecs, et documenter une procédure de bascule/dégradation si le CAS devient indisponible. Ces tests et mesures permettent d'assurer une intégration SSO fiable et sécurisée pour les utilisateurs et les équipes techniques.

Exemple de validation du ticket CAS :

```
const validateCasToken = async (ticket, serviceUrl) => {
  const casValidateUrl = `https://.../serviceValidate?service=${serviceUrl}&ticket=${ticket}`;
  const response = await axios.get(casValidateUrl);
  const userInfo = await parseCasResponse(response.data);
  return userInfo; // contient trigramme + infos CAS
};
```

4.3 Sécurité des données

Cette section détaille les mesures à appliquer pour protéger les **tokens**, garantir des communications chiffrées, restreindre l'accès aux API aux seuls utilisateurs autorisés (les "trigrammes") et assurer une journalisation utile et sûre. Le texte est structuré pour être compris par un lecteur non-technique tout en donnant des éléments concrets exploitables par une équipe technique.

4.3.1 Objectif

L'objectif principal de cette sous-section est d'assurer la confidentialité, l'intégrité et la traçabilité des échanges liés à l'authentification et à l'autorisation. Concrètement, il s'agit d'empêcher la capture et la réutilisation des jetons d'accès, de garantir la sécurité des canaux de communication, de n'autoriser que des identités explicitement habilitées et de conserver des journaux exploitables pour le diagnostic et la détection d'incidents sans exposer d'informations sensibles. Ces quatre axes structurent la politique de sécurité et orientent les choix d'implémentation (gestion des tokens, chiffrement, contrôle d'accès et logging).

4.3.2 Chiffrement et gestion des tokens

La stratégie retenue sépare deux types de jetons : les *access tokens* et les *refresh tokens*. Les access tokens sont de courte durée (typiquement 15 à 60 minutes) et sont signés (par exemple JWT RS256) afin de garantir leur intégrité et leur authenticité sans obligation d'interroger la base à chaque requête. Les refresh tokens ont une durée plus longue et sont gérés de manière stateful : ils sont stockés côté serveur (table `sessions` ou Redis) et **ne sont jamais conservés en clair** en base. Seul leur haché (HMAC-SHA256, bcrypt ou équivalent) est enregistré pour éviter que la fuite de la base ne permette de réutiliser les refresh tokens.

Il est important de distinguer signature et chiffrement : la signature vérifie que le token n'a pas été altéré, tandis que le chiffrement (JWE) rend son contenu illisible. Le chiffrement est conseillé si le payload contient des informations sensibles ; sinon la signature suffit. La gestion des clés (KMS / Vault) est cruciale : les clés de signature/chiffrement doivent être centralisées dans un gestionnaire de secrets, faire l'objet d'une rotation planifiée et d'une procédure de rollover assurant la rétrocompatibilité sur une courte fenêtre. Enfin, un mécanisme de révocation doit exister : stocker les identifiants de token (jti) révoqués dans Redis avec un TTL permet au middleware de vérifier une blacklist avant d'accepter un access token et ainsi d'invalidier immédiatement des sessions compromises.

4.3.3 HTTPS obligatoire

Toutes les communications sensibles doivent transiter exclusivement par TLS (TLS 1.2 minimum, TLS 1.3 recommandé) afin d'éviter toute interception ou modification en transit. La configuration des serveurs doit restreindre les suites de chiffrement aux algorithmes modernes offrant forward secrecy, activer HSTS pour forcer le passage en HTTPS côté navigateur et supporter des mécanismes d'OCSP stapling pour vérifier les certificats efficacement. La délivrance et le renouvellement des certificats doivent être automatisés (Let's Encrypt, ACM ou équivalent) et surveillés pour éviter expirations involontaires. Côté application, on force la redirection HTTP → HTTPS et on refuse toute exposition d'endpoints sensibles en clair ; dans les appels sortants (par exemple vers CAS) la validation stricte du certificat distant est impérative.

4.3.4 Contrôles d'accès — trigrammes et autorisation

Le contrôle d'accès repose sur la séparation nette entre authentification (vérifier l'identité) et autorisation (déterminer les droits). Dans notre modèle opérationnel, chaque identité est identifiée par un trigramme et dispose d'un statut et d'un ou plusieurs rôles. Le modèle recommandé combine RBAC (Role-Based Access Control) pour les cas d'usage courants et ABAC (Attribute-Based Access Control) lorsque des règles fines basées sur des attributs (département, service, horaires, etc.) sont nécessaires. La table `users` contient les trigrammes, le statut, les rôles et la liste des API autorisées ; un middleware d'autorisation, exécuté après le middleware d'authentification, vérifie `req.user.trigramme` et `req.user.roles` puis applique la règle du moindre privilège : aucune action n'est autorisée sans droit explicite. Enfin, le provisioning (création/modification) et la révocation des trigrammes doivent suivre un workflow formalisé (intégration AD/LDAP si pertinent) et être audités.

4.3.5 Journalisation et traçabilité

La journalisation doit fournir des informations suffisantes pour diagnostiquer des incidents et conduire des enquêtes tout en respectant la confidentialité. Les logs doivent contenir, pour chaque requête pertinente : horodatage, endpoint appelé, méthode HTTP, trigramme / userId (mais jamais le token), adresse IP source, user agent, statut HTTP retourné et motif d'échec (par exemple « token expiré »). Il est impératif de **ne pas** logger les tokens en clair, les mots de passe ni d'autres PII sensibles ; en milieu de staging, si un token doit être logué pour le debug, il doit être masqué. Les logs doivent être structurés (JSON) pour ingestion par un SIEM / ELK / Splunk, enrichis d'un `correlation_id` permettant de tracer une requête à travers les micro-services, et stockés de manière sécurisée avec chiffrement au repos. La politique de rétention doit être définie (par exemple 90 jours en accès rapide, archives chiffrées puis purge selon exigences réglementaires) et l'accès aux logs doit être restreint et audité.

4.3.6 Mesures d'atténuation et bonnes pratiques

Côté client web, il convient d'éviter le stockage des refresh tokens dans `localStorage` (risque XSS) : privilégier les cookies `HttpOnly`, `Secure`, et `SameSite`. Les protections contre les vulnérabilités XSS et CSRF doivent être appliquées (Content Security Policy, sanitation des inputs, tokens anti-CSRF si cookies utilisés). Les endpoints d'authentification doivent être protégés par des mécanismes de limitation de débit et d'anti-brute force et les adresses IP suspectes doivent être bloquées progressivement. Sur le plan organisationnel et qualité, des audits SAST/DAST réguliers et des revues de droits trimestrielles sont recommandés pour maintenir un niveau de sécurité adapté.

4.4 Développement du Frontend et gestion des événements

En parallèle du développement de l'API, j'ai conçu une application **frontend en React** permettant d'interagir avec les données stockées dans la base PostgreSQL via l'API sécurisée.

L'objectif principal était de fournir à l'équipe hardware une interface intuitive pour consulter, filtrer et gérer les ressources (workstations, processeurs, GPU, salles, logs, etc.), tout en respectant les mécanismes d'authentification CAS déjà implémentés côté backend.

Authentification côté frontend

Dès le chargement de l'application, le frontend vérifie la présence d'un **token d'authentification** dans l'URL ou dans le `localStorage`.

- Si aucun token n'est présent → l'utilisateur est redirigé vers le CAS Login.
- Si un token valide est présent → il est stocké et utilisé pour tous les appels API protégés.

Extrait du code : Ici, le **frontend dépend directement de l'API backend** pour valider le token auprès du CAS.

```

useEffect(() => {
  const checkAuth = async () => {
    const frontUrl = window.location.origin;

    // Récupération éventuelle du token en paramètre d'URL
    const urlParams = new URLSearchParams(window.location.search);
    const tokenInUrl = urlParams.get('token');
    const userInUrl = urlParams.get('username') ?? '';

    if (tokenInUrl) {
      localStorage.setItem('authToken', tokenInUrl);
      localStorage.setItem('username', userInUrl);
      localStorage.setItem('tokenTimestamp', Date.now().toString());
      window.history.replaceState({}, document.title, window.location.pathname);
    }

    const token = localStorage.getItem('authToken');
    if (!token) {
      // Pas connecté → redirection vers CAS
      window.location.href =
        `${API_URL}/api/login?redirectUrl=${encodeURIComponent(frontUrl)}`;
      return;
    }

    try {
      const { data } = await statsService.getActions();
      setActions(data.data || []);
      setUsername(localStorage.getItem('username') || '');
      setIsAuthorized(true);
    } catch (err) {
      console.error('Vérification du token échouée', err);
      localStorage.clear();
      window.location.href =
        `${API_URL}/api/login?redirectUrl=${encodeURIComponent(frontUrl)}`;
    } finally {
      setIsLoading(false);
    }
  };

  checkAuth();
}, []);

```

4.5 Gestion des événements et des données

Cette section décrit comment l'interface cliente gère le chargement, la mise à jour et l'affichage des données (entités et logs d'événements), ainsi que le maintien de l'état des filtres/recherches/tri sans recharger la page. Le texte est rédigé pour être compréhensible par un lecteur non-technique tout en fournissant des détails exploitables par l'équipe technique.

4.5.1 Objectif

L'objectif est d'offrir une expérience utilisateur réactive et cohérente : les vues doivent refléter l'état réel des ressources (workstations, GPU, salles, etc.) sans blocages inutiles, tout en évitant les requêtes réseau redondantes et les lectures de données obsolètes. Concrètement, cela implique que l'URL soit la source de vérité pour l'état de navigation/filtrage, que les effets côté client (`useEffect`) orchestrent les appels réseau de manière sûre (annulation, gestion de concurrence), et que les logs d'actions puissent être reçus en quasi-temps réel (polling ou WebSocket) avec une stratégie de cache adaptée.

4.5.2 Source de vérité : l'URL et la synchronisation d'état

Dans l'application, la route et les query parameters (par ex. `?room=lab1&status=available&page=2&sort=cpu`) constituent la source de vérité. Toute modification de filtre, de tri ou de page met à jour l'URL via `history.push/replace` : cela permet la pérennité des liens, le partage d'URLs et la navigation via l'historique du navigateur. Le composant lit l'URL au montage et chaque changement d'URL déclenche la logique de chargement des données. Cette conception garantit que l'état affiché est déterministe et ré-reproductible.

4.5.3 Cycle de vie d'un chargement (schéma `useEffect`)

Chaque chargement suit le même cycle logique. Le composant observe des dépendances (route, query params, token d'authentification, page). Au changement, un `useEffect` prépare une requête asynchrone : il crée un `AbortController` et associe son `signal` à la requête (`fetch/axios`) afin d'autoriser l'annulation si une nouvelle requête devient pertinente. L'UI passe en état `loading`, les erreurs antérieures sont effacées, la requête est lancée avec les bons en-têtes (ex. `Authorization: Bearer <token>`) et, à la réponse, l'état `data`, `loading` et `lastUpdated` sont mis à jour. En cas d'erreur, le code HTTP est interprété (401 → tentative de refresh → redirection si échec ; autres codes → affichage d'un message adapté). Si la requête est `aborted`, aucun état n'est appliqué — le nettoyage

reste silencieux. Le `return` du `useEffect` appelle systématiquement `controller.abort()` pour éviter les fuites et les races.

4.5.4 Annulation, concurrence et cohérence

Pour éviter les conditions de course et les états obsolètes, la règle est simple : avant d'appliquer une réponse réseau, s'assurer qu'elle correspond toujours à l'état courant (même route/params) et qu'elle n'a pas été annulée. L'usage d'un `AbortController` permet d'interrompre élégamment les requêtes précédentes. Pour des scénarios plus complexes (plusieurs requêtes concurrentes), on peut utiliser des clés (request keys) et vérifier que la réponse correspond à la clé la plus récente avant de mettre à jour le store local.

4.5.5 Gestion des états UI et indicateurs

L'UI maintient minimalement les états `loading`, `error`, `data` et quelques métadonnées utiles (`isStale`, `lastUpdated`, `total`, `page`, `per_page`). `isStale` signale que la donnée affichée provient d'un cache et qu'une réactualisation est en cours (stale-while-revalidate). Ces indicateurs nourrissent le composant pour afficher skeleta, messages d'erreur, badges "données mises à jour il y a X" ou boutons « rafraîchir ».

4.5.6 Cache court et stratégies de fetching

Pour limiter la latence et les requêtes redondantes, on recommande l'emploi d'un cache côté client (ex. React Query ou SWR). Ces bibliothèques fournissent : réutilisation des requêtes identiques, stratégie stale-while-revalidate, gestion centralisée des retries et backoff, et invalidation après mutation. Pour des données critiques ou volumineuses, conserver un cache mémoire court évite de recharger inutilement les mêmes listes lors des navigations cycles rapides. Pour les logs, on peut compléter par un cache persistant (IndexedDB) si l'objectif est la rétention en mode offline.

4.5.7 Pagination, scroll infini et UX

L'API doit renvoyer des métadonnées (`total`, `page`, `per_page`) pour piloter la pagination ou l'infinite scroll. En infinite scroll, lorsque l'utilisateur approche du bas, l'application incrémente `page` dans l'URL, ce qui déclenche le fetch de la page suivante. Lors de la concaténation des pages, il convient de détecter et dédupliquer les éléments (par ex. en utilisant un identifiant unique) pour éviter les doublons apparents. Pour des jeux de données très volumineux, déléguer tri/pagination au serveur reste préférable.

4.5.8 Comportement spécifique : logs d'actions (events)

Les logs d'actions sont volumineux et produits fréquemment ; deux approches sont possibles selon les contraintes :

Polling : un `useEffect` démarre un intervalle (ex. toutes les 10s) qui appelle `/api/events?since=<lastTimestamp>` pour récupérer les nouveaux événements. Le `cleanup` supprime l'intervalle. En cas d'erreur répétée, appliquer un backoff exponentiel pour éviter d'aggraver une situation de défaillance. Le polling doit être conçu pour récupérer seulement les nouveaux éléments (params `since/after`) afin d'être efficace.

WebSocket / push : lorsque la latence est critique, initialiser une connexion WebSocket authentifiée (`wss://...` avec token) dans un `useEffect`. Conserver la connexion au sein d'un contexte et la fermer au démontage. À la réception d'un événement, l'ajouter en tête de liste et — si besoin — persister en IndexedDB pour résilience offline. Le socket doit implémenter une logique de reconnexion progressive et une gestion des erreurs (auth failure → refresh token → redirection si nécessaire).

Dans les deux cas, le frontend expose des mécanismes de pagination/filtrage pour permettre la consultation historique et l'export éventuel.

4.5.9 Filtres, recherches et tris côté client (sans reload)

Les filtres et la recherche sont synchronisés avec la query string. Modifier un filtre met à jour l'URL sans recharger la page, déclenchant le `useEffect` associé. Pour les champs texte, appliquer un `debounce` (300–500 ms) évite d'émettre une requête à chaque frappe et améliore la réactivité. Les mutations utilisateur (par ex. attribuer une workstation) utilisent des mises à jour optimistes : l'interface reflète immédiatement le changement, la requête API est envoyée en arrière-plan et, en cas d'échec, l'état est rollbacké et un message d'erreur est affiché. Le tri et la pagination doivent être délégués au serveur lorsque le volume le justifie ; pour de petits ensembles, un tri local est acceptable.

4.5.10 Sécurité et résilience des appels

Toutes les requêtes côté client doivent inclure les en-têtes d'authentification gérés par un intercepteur (axios/fetch wrapper). Les erreurs 401/403 doivent déclencher une logique centralisée : tentative de refresh via refresh token, sinon redirection vers le flux d'authentification (CAS). Les timeouts raisonnables et la gestion d'erreur explicite (affichage d'un message utilisateur, journalisation côté client pour les erreurs non critiques) améliorent la robustesse.

Exemple de chargement dynamique des entités :

```

// Chargement des données selon l'entité consultée
const loadEntityData = async (entityType: string) => {
  try {
    let response;
    switch (entityType) {
      case 'workstations':
        response = await statsService.getWorkstations();
        break;
      case 'gpus':
        response = await statsService.getGpus();
        break;
      case 'processors':
        response = await statsService.getProcessors();
        break;
      case 'rooms':
        response = await statsService.getRooms();
        break;
      case 'owners':
        response = await statsService.getOwners();
        break;
    }
    if (response && response.data) {
      setEntityData(response.data.data);
    }
  } catch (error) {
    console.error('Error loading data:', error);
  }
};

// Déclencheur automatique à chaque changement de route
useEffect(() => {
  const path = window.location.pathname;
  const entityType = path.substring(1);
  if (entityType && ['workstations', 'gpus', 'processors', 'rooms', 'owners'].includes(entityType))
    loadEntityData(entityType);
}, [window.location.pathname]);

```

Intégration avec l'API et gestion des logs

Toutes les actions utilisateur (consultation, ajout, modification, suppression) sont envoyées vers l'API.

L'API, de son côté, enregistre un **log horodaté** en base, associé au trigramme de

l'utilisateur. Exemple : lors de la consultation des logs, le frontend envoie une requête protégée

V. Bilan professionnel

La deuxième année d'apprentissage chez Dassault Systèmes se conclut par un bilan globalement très positif : les objectifs opérationnels définis au lancement du projet — migration des fichiers Excel vers une base PostgreSQL structurée, mise en place d'une API REST Node.js/Express pour les opérations CRUD, intégration d'un SSO via CAS avec création de sessions internes, développement d'un middleware d'authentification et prototypage d'une interface React pour consultation et filtrage — ont été atteints et fonctionnent concrètement, apportant des gains tangibles en termes d'intégrité des données, de centralisation et d'expérience utilisateur. Ces réalisations témoignent d'une maîtrise technique solide (backend asynchrone en Node/Express, ORM Sequelize pour PostgreSQL, parsing CAS via xml2js, chiffrement et gestion basique des tokens via crypto), d'une méthodologie pragmatique (approche itérative, documentation, usage de Postman pour les tests manuels, CI/CD via GitLab) et d'une forte capacité d'adaptation aux contraintes métier (utilisation de ngrok pour tests HTTPS, échanges fréquents avec le tuteur et utilisateurs finaux). Par ailleurs, le prototype React, bien que perfectible, apporte déjà une valeur ajoutée immédiate : synchronisation URL ↔ filtres, debounce sur les recherches et ergonomie de base permettant de remplacer progressivement les processus Excel lourds et error-prone. Cela a permis une appropriation rapide par l'équipe hardware et une réduction visible des manipulations manuelles. Néanmoins, l'analyse critique met en lumière des domaines d'industrialisation prioritaires — non pas comme des échecs, mais comme des leviers d'amélioration clairement identifiés et réalisables : la couverture des tests automatisés (unitaires et d'intégration) reste insuffisante et doit être renforcée pour réduire le risque de régressions ; la gestion fine du cycle de vie des sessions (refresh tokens, révocation via jti, Single Logout) requiert une implémentation robuste (table sessions / blacklist Redis, endpoints d'administration) pour garantir une réponse immédiate en cas d'incident de sécurité ; la gouvernance des secrets et des clés de signature/chiffrement (KMS, rotation documentée) doit être formalisée pour répondre aux exigences de conformité ; la journalisation, déjà présente, doit évoluer vers des logs structurés (JSON) enrichis d'un `correlation_id` et intégrés dans une chaîne SIEM/ELK afin d'accélérer le diagnostic et l'investigation ; enfin, la migration des données Excel nécessite des scripts idempotents de nettoyage et de validation pour assurer la qualité et l'historique des informations sans interventions manuelles récurrentes. Ces points proviennent moins d'un manque de

compétence que d'une priorisation salariale orientée vers la livraison fonctionnelle rapide — une décision logique au stade prototypal — et qui ouvre maintenant la phase naturelle d'industrialisation. Pour traduire cette volonté en actions opérationnelles, il est pertinent de prioriser à court terme l'automatisation des tests critiques et l'intégration CI, l'implémentation d'un mécanisme de révocation instantanée des sessions, la structuration des logs et la mise en place de politiques de gestion des secrets ; à moyen terme, automatiser et rendre idempotentes les migrations depuis Excel, adopter un cache client tel que React Query ou SWR pour améliorer la latence et l'UX, et déployer des tests E2E et de charge ; à long terme, formaliser les procédures de bascule/SLO pour CAS et planifier des audits SAST/DAST. Ces chantiers sont réalistes au regard des acquis techniques et de l'organisation en place, et permettront de transformer un prototype fonctionnel et utile en un service industriel, résilient et conforme. Sur le plan personnel, cette année a considérablement renforcé mes compétences en authentification et sécurité, en industrialisation (CI/CD, tests, observabilité) et en coordination transverse avec les utilisateurs — des acquis qui me confèrent désormais une crédibilité pour porter les améliorations identifiées et encadrer des travaux plus stratégiques à venir. En synthèse, le projet est une réussite concrète : il a remplacé un processus Excel fragile par une plateforme centralisée et utilisable, tout en identifiant des axes d'amélioration pragmatiques et prioritaires qui, une fois traités, permettront d'atteindre un niveau de maturité opérationnelle et de sécurité pleinement compatible avec les exigences de Dassault Systèmes.

VI. Conclusion

Cette deuxième année d'apprentissage a confirmé et enrichi le travail initié l'an dernier. Alors que ma première année était centrée sur la **conception d'une base, d'une API et d'un prototype frontend**, cette année m'a permis d'aller plus loin en **sécurisant et industrialisant** la solution : intégration du **CAS**, chiffrement des tokens, gestion des droits, mise en place de la **journalisation** et amélioration significative de l'interface utilisateur.

Ces avancées techniques m'ont sensibilisé à des enjeux essentiels pour une entreprise de la taille de Dassault Systèmes : la **sécurité des données**, la **traçabilité des actions** et l'**expérience utilisateur**. Elles m'ont aussi donné une plus grande autonomie et une confiance renforcée dans mes capacités à mener des projets à fort impact.

En conclusion, cette deuxième année a représenté un tournant : je suis passé de la **phase de prototypage** à la **phase de fiabilisation et d'intégration**, ce qui reflète bien ma progression en tant qu'apprenti ingénieur. Je suis désormais enthousiaste à l'idée d'aborder

ma troisième année avec de **nouveaux projets ambitieux**, comme la certification logicielle de particuliers, qui me permettront de continuer à évoluer et de confirmer ma place au sein de l'entreprise.

Bibliographie

History – Dassault Systèmes

<https://www.3ds.com/about/company/history>

Dassault Systèmes (page Wikipedia en anglais)

https://en.wikipedia.org/wiki/Dassault_Syst%C3%A8mes

Dassault Systèmes – chiffres clés et présentation

<https://dassault.fr/en/subsidiary/dassault-systemes>

React (site officiel)

<https://react.dev/>

React (page Wikipedia)

https://en.wikipedia.org/wiki/React_%28software%29

Apereo – Central Authentication Service (CAS)

<https://www.apereo.org/programs/software/cas>

CAS (page Wikipédia)

https://en.wikipedia.org/wiki/Central_Authentication_Service

PostgreSQL (page Wikipedia)

<https://en.wikipedia.org/wiki/PostgreSQL>

PostgreSQL Wiki (documentation communautaire)

https://wiki.postgresql.org/wiki/Main_Page

Sequelize (ORM for Node.js) — <https://sequelize.org/>

axios (HTTP client) — <https://axios-http.com/>

xml2js (npm package) — <https://www.npmjs.com/package/xml2js>

Node.js crypto (module natif) — <https://nodejs.org/api/crypto.html>

Résumé

Durant ma deuxième année d'apprentissage chez Dassault Systèmes, j'ai transformé un processus de certification fragile, reposant sur des fichiers Excel, en une application web sécurisée et opérationnelle, tout en renforçant mes compétences techniques et organisationnelles. Les objectifs du projet étaient de migrer les données de certification vers PostgreSQL, d'exposer une API REST en Node.js/Express pour les opérations CRUD, de sécuriser l'accès via le SSO CAS et des tokens internes, de valider les saisies et de journaliser les actions, et enfin de fournir une interface utilisateur en React.

Sur le plan technique, j'ai défini un schéma PostgreSQL et réalisé la migration des données, développé un backend Express avec un middleware qui valide les tokens et injecte `req.user`, implémenté la validation des tickets CAS (serviceValidate → session/token interne) ainsi qu'un cycle de vie basique des tokens (génération, stockage, usage via en-tête Authorization). Côté frontend, j'ai construit un prototype React qui synchronise filtres et pagination avec l'URL, utilise des recherches débouçées, applique des mises à jour optimistes pour l'expérience utilisateur et s'appuie sur des hooks `useEffect` pour récupérer les entités et les événements (avec `AbortController` pour l'annulation). Pour les logs/événements, j'ai mis en place un mécanisme de polling et conçu l'architecture pour accepter une solution push via WebSocket si nécessaire. Les bibliothèques et outils clés employés sont : React, Node.js, Sequelize, axios, xml2js, le module natif `crypto` de Node, Postman, GitLab CI/CD et ngrok pour les tests HTTPS.

Le projet a produit des gains concrets : meilleure intégrité des données, centralisation des accès, un flux SSO fonctionnel pour les cas standards, une API soutenant l'interface et une interface React utilisable qui remplace de nombreuses opérations manuelles sur Excel. J'ai adopté une démarche itérative — documentation, tests manuels et unitaires, et retours réguliers avec mon tuteur et les utilisateurs finaux — ce qui a accéléré l'adoption et permis d'aligner rapidement les livrables sur les besoins.

Les écarts restants concernent principalement l'industrialisation et le durcissement plutôt que les fonctionnalités de base : la couverture des tests automatisés et les pipelines CI doivent être étendus ; la révocation des sessions (jti / blacklist), le Single Logout (déconnexion centralisée) et la gestion des modes dégradés pour CAS restent à finaliser ; la gestion des secrets et des clés (KMS, rotation) doit être formalisée ; les logs doivent être structurés (JSON) et corrélés pour ingestion SIEM ; enfin, la migration Excel → base nécessite des scripts de nettoyage idempotents. Ce sont des étapes prioritaires pour la suite.

Sur le plan personnel, cette année a fortement consolidé mes compétences en authentification (CAS, cycle de vie des tokens, cryptographie), en journalisation et observabilité, en patterns d'intégration frontend, ainsi qu'en collaboration transverse. Pour l'année à venir, je compte concentrer mes efforts sur la mise à maturité du système vers un environnement de production : ajout de tests automatisés, implémentation de la révocation des sessions, meilleure gestion des secrets, mise en place d'un cache côté client (React Query / SWR), automatisation des migrations et amélioration de l'observabilité — afin que le prototype devienne un service résilient, sécurisé et maintenable au sein de Dassault Systèmes.

Résumé anglais

During my second year as an apprentice at Dassault Systèmes I led the transformation of a fragile, Excel-based certification process into a working, secure web application while significantly advancing my technical and organizational skills. The mission addressed a clear business need: migrate scattered spreadsheets into a centralized, auditable system to improve data integrity, reduce manual errors, secure access with corporate SSO (CAS), and provide an intuitive interface for the hardware team to consult and manage resources (workstations, GPUs, processors, rooms, and certification logs).

The project objectives were concrete and measurable: design a PostgreSQL schema and migrate Excel data; expose a Node.js/Express REST API for CRUD operations; implement a middleware integrating CAS and internal tokens for authentication and authorization; enforce server-side validation and data constraints; log all operations for traceability; and prototype a React frontend that synchronizes filters with the URL and minimizes page reloads. I adopted an iterative approach—small, testable deliveries, documentation, and frequent feedback with my tutor and end users—which allowed the team to validate assumptions early and adjust priorities.

Technically, I implemented a relational database model in PostgreSQL to replace ad-hoc Excel tables, introducing uniqueness constraints and referential integrity to avoid duplication and anomalies. On the backend, I built a modular Express API using Sequelize as the ORM to interact with PostgreSQL. The authentication flow relies on CAS for SSO: the front-end redirects unauthenticated users to CAS, the backend validates the returned ticket (serviceValidate), and then issues a short-lived internal token (or session) stored securely. Middleware centralizes token validation and injects `req.user` so handlers can perform authorization checks based on trigramme and role. Key libraries and tools used include axios for HTTP interactions, xml2js for parsing CAS XML responses, Node's native crypto module for token operations, Postman for API testing, GitLab for source control and CI/CD, and ngrok to expose HTTPS endpoints during development.

On the frontend I developed a React prototype focused on practical UX: URL-driven state (filters, page, sort) for shareable views, debounced search inputs (300–500 ms) to reduce unnecessary requests, optimistic UI updates for immediate feedback after mutations, and useEffect hooks orchestrating data fetching. Each useEffect follows a robust pattern: derive dependencies from route and query params, create an AbortController to cancel stale requests, set loading/error states, call the API with Authorization headers, and cleanup on unmount. For logs and events I implemented polling (configurable interval) and designed the architecture to accept WebSocket push as an alternative for lower latency. I also introduced

deduplication when concatenating paginated results and recommended client-side caching (React Query / SWR) to implement stale-while-revalidate behavior and minimize redundant calls.

The delivered solution already produces tangible gains: improved data integrity and consistency versus Excel, centralized access and basic traceability of certification actions, a working SSO flow for standard cases, and a front-end that reduces manual operations for the hardware team. The iterative delivery model enabled early adoption and useful user feedback that shaped subsequent refinements.

That said, the project also surfaced operational gaps that I have treated as priority next steps rather than failures. Test automation coverage needs to be broadened: unit tests for core middleware, integration tests for CAS ticket validation, and E2E scenarios for the main user journeys must be added to the CI pipeline to reduce regression risk. Session lifecycle handling requires hardening: implement a sessions table and jti blacklist (Redis) for immediate revocation, complete refresh-token logic, and test Single Logout (SLO) scenarios. Security practices must be formalized—migrate secrets and signing keys to a KMS or Vault, document and schedule key rotation, and ensure cookies are HttpOnly/Secure/SameSite where applicable. Observability also warrants investment: structure logs in JSON with `correlation_id` for SIEM ingestion, collect basic metrics (API error rates, latency, cache hit ratio), and set retention/encryption policies for logs. Finally, the Excel→DB migration needs idempotent scripts and validation reports to avoid manual cleanup and to preserve historical fidelity.

I translated these findings into a prioritized action plan: immediately add tests for auth middleware and CAS flows, implement session revocation and admin endpoints, and instrument logs and metrics; in the medium term, automate migration scripts, integrate React Query for client caching to improve UX and reduce latency, and set up E2E and load tests; in the longer term, formalize CAS SLO procedures, schedule SAST/DAST audits, and deploy production-grade secrets management.

From a personal standpoint, this year dramatically increased my competence in authentication and security (CAS, token lifecycle, crypto), backend and database design (Node.js, Express, Sequelize, PostgreSQL), and frontend patterns for real-world apps (React hooks, optimistic updates, URL-driven state). It also improved my ability to bridge technical work with user needs—collaborating with end users, prioritizing features that reduce manual effort, and documenting operational practices for maintainers. The experience strengthened my confidence and credibility within the team and prepared me to lead the next phase: maturing the prototype into a resilient, observable, and auditable service compatible with Dassault Systèmes' operational and security standards.

In conclusion, the year delivered a functional, valuable platform that replaces error-prone Excel processes with a centralized, secure application; it also produced a clear, actionable roadmap to reach industrial maturity. My immediate focus for the coming months will be to raise test coverage, implement session revocation and secrets management, improve observability, and finalize automated migration—steps that will transform the current prototype into a production-ready service with high reliability and maintainability.